

Proyecto de Curso: Análisis, Diseño y Construcción de un Sistema Operativo desde Cero

Por

Castro Pari, Rayneld Fidel

Mamani Flores, Natan

Mendoza Quispe, Jose Daniel

Polo Chura, Marco Rosauro

Trabajo académico presentado a la

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

como

Proyecto de Investigación de la Primera Unidad de Sistemas Operativos



Departamento Académico de Ingeniería de Informática y de Sistemas

Asesor: Ugarte Rojas, Héctor Eduardo

Memorial Universidad Nacional San Antonio Abad del Cusco

Semestre 2025-II

Cusco

Perú

Índice general

Índice de tablas

Índice de figuras

Introduccion

El desarrollo y la transformación de los sistemas operativos han sido fundamentales en la evolución de las tecnologías de la información. Desde los primeros programas creados para las computadoras centrales hasta los sistemas más utilizados en ordenadores personales, dispositivos móviles y equipos embebidos, los sistemas operativos han actuado como un puente entre las necesidades del usuario y la complejidad del hardware. Estudiarlos no solo permite entender el funcionamiento básico de una computadora, sino que también brinda las bases conceptuales y técnicas necesarias para el diseño de nuevas soluciones informáticas. Este documento tiene como finalidad ofrecer una perspectiva global de los sistemas operativos, abordando sus características principales, su arquitectura y sus componentes.

Objetivo de la investigacion

El objetivo principal de este trabajo es realizar un análisis técnico y comparativo de distintos sistemas operativos, con la finalidad de identificar sus características más relevantes y examinar las diversas concepciones y enfoques aplicados en su diseño e implementación. Se estudiarán tanto arquitecturas tradicionales, como la monolítica,

así como modelos más modernos, entre ellos los exokernels, híbridos y microkernels . Del mismo modo, se pretende ofrecer una visión crítica sobre las ventajas, limitaciones y ámbitos de aplicación de cada sistema operativo.

Alcance del análisis

El presente estudio se centra en sistemas operativos ampliamente utilizados y representativos de diferentes enfoques arquitectónicos y áreas de aplicación. No se abordarán sistemas altamente experimentales o versiones obsoletas a menos que sean relevantes para la comparación histórica o conceptual. El análisis incluye tanto sistemas de propósito general (Windows, macOS, Linux) como sistemas especializados (RTOS, embebidos y móviles), enfocándose en aspectos técnicos, operativos y comunitarios que permitan identificar fortalezas, limitaciones y escenarios de uso recomendados para cada sistema operativo.

Metodologia

Con el fin de alcanzar los objetivos planteados, se seguirá una metodología compuesta por tres fases fundamentales:

1. **Revisión bibliográfica y documental:** Se consultarán libros especializados, artículos académicos, blogs técnicos, documentación oficial y repositorios relacionados con los sistemas operativos en estudio.
2. **Evaluación técnica:** Se examinarán los aspectos esenciales de cada sistema, incluyendo la gestión de procesos, la administración de memoria, los sistemas de

archivos, el manejo de dispositivos, las interfaces de usuario y los mecanismos de seguridad.

3. **Comparación estructurada:** Se elaborará una tabla que sintetice las características más representativas de los sistemas operativos analizados, tomando en cuenta factores como el tamaño del kernel, la arquitectura, la comunidad, el lenguaje de implementación, la documentación disponible y el soporte de hardware.

Este enfoque permitirá garantizar un análisis más objetivo, completo y útil, tanto para fines académicos como prácticos.

Capítulo 1

Propuestas analizadas

1.1 XV6

1.1.1 Nombre del proyecto o sistema operativo

El sistema operativo xv6 es una versión moderna y simplificada del Unix Sexta Edición (V6), creada con fines educativos. Fue desarrollado por el MIT en 2006 para el curso de Engineering Operating Systems (6.828) (?), con el objetivo de ofrecer una herramienta didáctica más accesible y actual que el Unix original, el cual estaba escrito en un dialecto antiguo de C (pre-ANSI). Esta reimplementación mantiene la esencia y estructura del Unix clásico, pero utiliza un código más claro y compatible con los entornos de aprendizaje contemporáneos.

1.1.2 Enlace al repositorio y/o documentación oficial

- Pagina oficial: <https://pdos.csail.mit.edu/6.828/2018/xv6.html>
- Enlace al repositorio oficial: <https://github.com/mit-pdos/xv6-public>
- Enlace al libro oficial: <https://pdos.csail.mit.edu/6.828/2018/xv6/book-rev10.pdf>

1.1.3 Objetivo del proyecto

El propósito principal de xv6 es educativo y experimental. Fue diseñado para funcionar como un kernel de estudio que permita a los estudiantes comprender los principios fundamentales de Unix dentro de un curso semestral (?). A diferencia de sistemas operativos más complejos como Linux o BSD, xv6 mantiene una estructura lo bastante simple como para ser analizada y entendida en su totalidad durante una

asignatura, pero al mismo tiempo conserva los componentes esenciales y la lógica interna de un sistema Unix real.

1.1.4 Lenguaje(s) de implementación

El núcleo de xv6 está escrito principalmente en ANSI C, con pequeñas porciones en lenguaje ensamblador (por ejemplo, para rutinas de arranque y manejo de interrupciones) (? , pag. 7),. La versión original de x86 es C (para x86-32); la versión reciente soporta RISC-V también escrita en C

1.1.5 Arquitectura del sistema (monolítica, microkernel, etc.)

Como explica la documentación, xv6 es un kernel monolítico: toda la funcionalidad del sistema corre con privilegios de kernel, sin servidores externos (? , pag. 25). En Xv6, el kernel sigue el diseño clásico de los sistemas Unix. Funciona como un núcleo único que brinda servicios a múltiples procesos, los cuales se ejecutan en un espacio de usuario independiente. Cada proceso va alternando entre dos modos de ejecución:

- **En el modo usuario:** el proceso ejecuta su propio código y realiza operaciones computacionales básicas.
- **En el modo kernel:** es el sistema operativo quien toma el control para ejecutar las llamadas al sistema solicitadas.

Este cambio ocurre cuando un proceso necesita un servicio del sistema operativo: el hardware cambia automáticamente a un modo de mayor privilegio (modo supervisor) y transfiere la ejecución al kernel. Una vez que este completa la tarea, devuelve el control al proceso en el espacio de usuario. (? , pag. 9),

1.1.6 Componentes implementados (procesos, memoria, archivos, etc.)

Xv6 implementa los componentes esenciales de un sistema operativo Unix clásico:

Tabla 1.1: Componentes principales del sistema operativo **XV6**

Componente / Sub-sistema	Archivo(s) principal(es)	Funcionamiento técnico (idea clave)
Gestión de procesos y planificador	<code>proc.c</code> , <code>proc.h</code>	Define la estructura <code>proc</code> y el planificador round-robin por CPU; maneja el ciclo de vida de procesos (<code>fork</code> , <code>exec</code> , <code>exit</code> , <code>wait</code>).
Gestión de memoria virtual	<code>vm.c</code> , <code>kalloc.c</code>	Implementa tablas de páginas por proceso (Sv39) y asignador físico mediante <code>kalloc/kfree</code> .
Sistema de archivos	<code>fs.c</code> , <code>file.c</code> , <code>log.c</code>	FS tipo Unix con <code>inode</code> , directorios, caché de bloques y journaling mediante write-ahead log.
Interfaz de llamadas (syscalls)	<code>syscall.c</code> , <code>sysproc.c</code> , <code>usys.S</code>	Tabla de <code>syscalls</code> y despacho de traps del modo usuario al kernel (<code>fork</code> , <code>read</code> , <code>write</code> , etc.).
subsistemas de E/S y drivers	<code>console.c</code> , <code>uart.c</code> , <code>virtio_disk.c</code>	Drivers básicos UART y VirtIO integrados con la caché de bloques.
Comunicación entre procesos (IPC)	<code>pipe.c</code> , <code>sys_pipe.c</code>	Implementa pipes unidireccionales en memoria; permite transmisión de datos entre procesos relacionados mediante <code>read/write</code> sincronizados.

Gestión de procesos y planificación

La creación de procesos usa `fork()` para copiar el proceso padre y `exec()` para cargar nuevos programas. La planificación emplea un algoritmo **round-robin** básico, donde cada CPU asigna turnos de tiempo fijo a los procesos en una lista cíclica, sin prioridades. Todos los procesos comparten el mismo entorno básico, sin distinción entre usuarios (?, pag. 82).

Gestión de memoria virtual

Xv6 gestiona la memoria mediante paginación, asignando a cada proceso un espacio de direcciones virtual independiente. Utiliza solo 38 bits de los 64 que soporta RISC-V, permitiendo teóricamente hasta 256 GB de memoria por proceso (?, pag. 38).

El espacio de direcciones de cada proceso sigue la convención Unix clásica: La distribución de la memoria sigue el diseño clásico de Unix:

- **Segmento de Código:** Ubicado en las direcciones más bajas (cerca de 0), con permisos de solo lectura y ejecución para proteger el código del programa.
- **Datos y Heap:** Situados a continuación del código, con permisos de lectura y escritura. El heap puede expandirse dinámicamente cuando el programa solicita más memoria mediante la llamada `sbrk`.
- **Pila del Usuario:** Ubicada en la parte superior de la memoria, crece hacia abajo y tiene una *página guardia* inferior. Cualquier acceso a esta página causa que el kernel termine el proceso, previniendo desbordamientos.

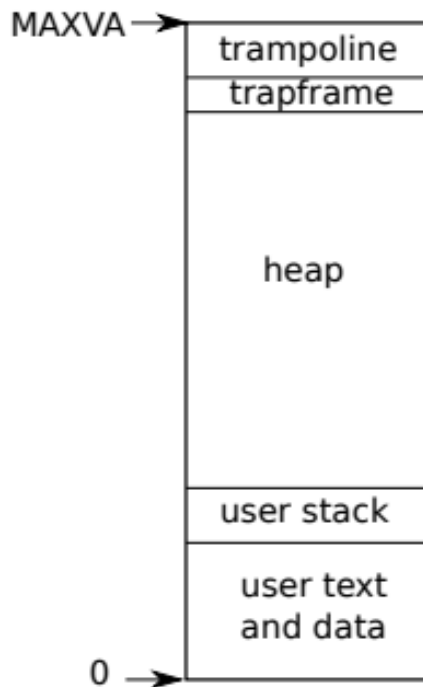


Figura 1.1: Disposición del espacio de direcciones virtuales de un proceso.

Fuente: Obtenido de (?) *xv6: a simple, Unix-like teaching operating system*.

Sistema de archivos

Xv6 implementa un sistema de archivos jerárquico que garantiza la integridad de los datos mediante journaling. Su diseño por capas organiza las funciones desde el acceso al disco hasta la gestión de archivos, haciendo que cada componente sea especializado y manejable. (?).

Organización en Disco El disco se estructura en secciones bien definidas:

- **Bloque 0:** Contiene el código de arranque
- **Bloque 1:** Almacena el “superbloque” con los metadatos del sistema de archivos
- **Bloques siguientes:** Albergan los inodos (que representan archivos)

- **Secciones posteriores:** Incluyen mapas de bits para espacios libres y los bloques de datos reales
- **Zona final:** Reservada para el journal o registro transaccional

Gestión de Archivos y Directorios Los directorios son archivos especiales que asocian nombres con inodos. Las rutas se resuelven secuencialmente (ej: `/usr/bin/ls` busca `usr`, luego `bin`).

Las operaciones básicas (`open`, `read`, `write`, `close`) usan **descriptores de archivo** y una **caché de bloques**. `read` carga datos a memoria, mientras `write` los marca para escritura transaccional. (? , pag. 85-86).

Interfaz de llamadas (syscalls)

Xv6 implementa llamadas al sistema que siguen el modelo tradicional de Unix, permitiendo a los programas de usuario solicitar servicios del kernel mediante interrupciones de software.(? , pag. 8-9).

El mecanismo funciona de la siguiente manera:

1. Un proceso en modo usuario invoca una syscall, lo que hace que el hardware genere una interrupción.
2. Esta interrupción transfiere el control al kernel, cambiando el modo de ejecución de la CPU.
3. El kernel identifica la syscall solicitada a través de un número único y ejecuta la función correspondiente.

4. Una vez completada, el kernel devuelve el control al proceso de usuario con los resultados necesarios.

E/S y drivers

Los dispositivos en xv6 se integran en el sistema bajo el principio de que “todo es un archivo”. Esto se logra mediante **archivos de dispositivo**, creados con la llamada al sistema `mknod`.

- `mknod(path, major, minor)`: Crea un archivo especial que representa un dispositivo. Los parámetros `major` (mayor) y `minor` (menor) son números que identifican de forma única al controlador (*driver*) del kernel y al dispositivo específico.

Controladores de Dispositivos (Drivers) El kernel incluye controladores mínimos pero funcionales para hardware esencial. Algunos ejemplos son:

- `uart.c` para el puerto serie.
- `virtio_disk.c` para el acceso al disco.
- `timer.c` para el temporizador del sistema.

Comunicación entre procesos (IPC)

Xv6 implementa un mecanismo básico de comunicación entre procesos (IPC) mediante **pipes**. Las pipes son canales unidireccionales en memoria que permiten a dos procesos relacionados intercambiar datos de forma sincronizada. (? , pag. 73–74),

1.1.7 Herramientas utilizadas (compiladores, emuladores, etc.)

Para la construcción, compilación y ejecución de *xv6*, se emplean las siguientes herramientas principales:

- **GCC (GNU C Compiler):** Compila el kernel y los programas de usuario escritos en C, generando binarios ELF compatibles con las arquitecturas **x86-32** o **RISC-V**.
- **QEMU:** Emulador de hardware ampliamente utilizado para ejecutar *xv6* en entornos virtuales. Permite probar el sistema operativo sin necesidad de hardware físico, simulando arquitecturas **x86** o **RISC-V**.
- **Cross-compiler:** En sistemas como macOS o Windows, se requiere un compilador cruzado para generar binarios ELF destinados a la arquitectura objetivo de *xv6*.

(?),

1.1.8 Nivel de complejidad y accesibilidad para estudiantes

Xv6 fue diseñado específicamente para ser un sistema operativo ligero y educativo. Su código completo consta de aproximadamente 10,000 líneas (equivalente a unas 99 páginas impresas) Su simplicidad relativa (ausencia de capas complejas como módulos dinámicos) lo hace muy accesible para estudiantes de OS. Aun así, cubre los conceptos esenciales de concurrencia, paginación, sistemas de archivos y llamadas al sistema típicos de un Unix real. (?, pag. 10–11).

1.2 RedoxOS

1.2.1 Nombre del proyecto o sistema operativo

El sistema operativo examinado es "Redox OS". De acuerdo con (?), su denominación deriva de la palabra "Redox", que es la abreviatura de "*reduction-oxidation*", un término de química que representa el intercambio equilibrado de electrones; esto refleja la meta del proyecto: alcanzar un sistema equilibrado en seguridad, rendimiento y simplicidad, a través de un diseño contemporáneo y fiable, redactado totalmente en Rust.

1.2.2 Enlace al repositorio y/o documentación oficial

- Enlace al repositorio oficial: <https://github.com/redox-os>
- Enlace al libro oficial: <https://doc.redox-os.org/book/>

1.2.3 Objetivo del proyecto

Según (??), Redox OS tiene un enfoque fundamentalmente educativo y experimental, pero también pretende ofrecer una opción práctica y segura en comparación con sistemas Unix tradicionales. Su objetivo es probar que un sistema operativo que esté totalmente desarrollado en Rust puede lograr la misma estabilidad y eficiencia que los sistemas comerciales, mientras elimina categorías completas de errores relacionados con la memoria y la concurrencia.

1.2.4 Lenguaje(s) de implementación

Redox OS está predominantemente desarrollado en Rust, abarcando su kernel, controladores, gestor de memoria, sistema de archivos y gran parte de las herramientas de usuario.

Ciertos elementos específicos, como el cargador de arranque y aspectos de la compatibilidad con C, emplean "ensamblador y C" de manera restringida.

1.2.5 Arquitectura del sistema

Redox OS adopta una arquitectura híbrida de microkernel, inspirada en el diseño de Minix 3 y el modelo de Unix, pero con un enfoque mucho más fuerte en la seguridad y la modularidad. El kernel de Redox es extremadamente pequeño: se encarga solo de la planificación, la gestión básica de memoria y la comunicación entre procesos. Todos los servicios del sistema, como el manejo de archivos, controladores, red y entorno gráfico, se ejecutan en el espacio de usuario como procesos independientes llamados schemes.

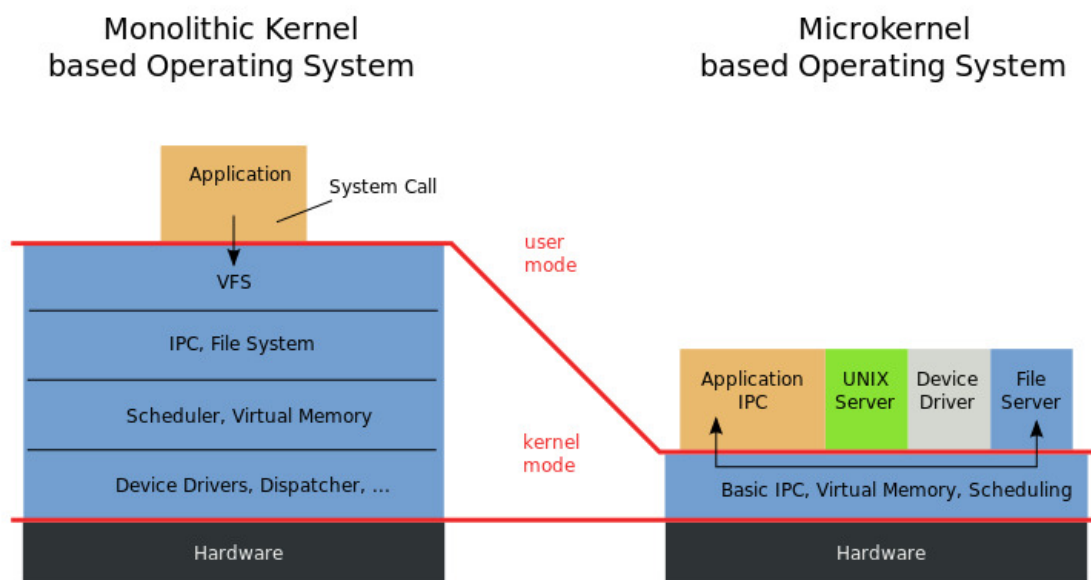


Figura 1.2: Comparativa de la arquitectura de Redox frente a arquitecturas tradicionales.

Fuente: Obtenido de (?).

1.2.6 Componentes implementados

En Redox OS, los componentes principales del sistema se implementan como crates escritos en Rust, siguiendo una arquitectura microkernel. Cada crate encapsula un servicio o subsistema (procesos, memoria, sistema de archivos, E/S, etc.) con una estricta separación de privilegios. El microkernel se encarga de la comunicación por mensajes entre estos servicios, garantizando seguridad, aislamiento y estabilidad.

A continuación, se presenta una tabla con los principales componentes y subsistemas de Redox OS, los nombres de sus crates en el repositorio oficial y la idea técnica detrás de su funcionamiento.

Tabla 1.2: Funcionamiento técnico de los principales subsistemas de Redox OS

Componente / Sub-sistema	Nombre en Redox (archivo/crate)	Funcionamiento técnico (idea clave)
Gestión de memoria	<code>memory</code> , <code>paging</code> , <code>rmm</code>	Maneja paginación virtual y asignación segura de marcos físicos.
Gestión de procesos y multitarea	<code>kernel</code> , <code>scheme::proc</code>	Ejecuta procesos aislados con planificación cooperativa.
Sistema de archivos	<code>redoxfs</code>	Sistema nativo en Rust con soporte concurrente.
Subsistemas de E/S y drivers	<code>drivers</code> , <code>schemes</code>	Drivers en espacio de usuario con namespaces restringidos.
Comunicación entre procesos (IPC)	<code>syscall</code> , <code>scheme</code> , <code>libredox</code>	IPC basado en mensajes mediante estructuras SQE/CQE.
Planificación y temporización	<code>scheduler</code> , <code>timer</code>	Planificador por prioridad con temporizador interno.
Gestión de usuarios y permisos	<code>user</code> , <code>authd</code>	Controla autenticación y separación de privilegios.
Interfaz gráfica (GUI)	<code>orbital</code> , <code>orbtk</code>	Servidor de ventanas y toolkit gráfico modular.
Red y conectividad	<code>netstack</code> , <code>ethernetd</code> , <code>ipd</code>	Pila de red completa en espacio de usuario.
Gestión del sistema y servicios	<code>init</code> , <code>logd</code> , <code>backgroundd</code>	Controla inicio, logs y servicios del sistema.
Bibliotecas del sistema y compatibilidad	<code>relibc</code> , <code>redox_rt</code> 17	Implementan llamadas POSIX sobre los <i>schemes</i> .

Fuente: Elaboración propia con base en (??).

Gestión de memoria

La gestión de memoria en Redox OS utiliza paginación virtual y asignación segura de marcos físicos. El crate `rmm` implementa el gestor de memoria del kernel, garantizando aislamiento entre procesos y eficiencia en la asignación dinámica (?).

Gestión de procesos y multitarea

El kernel de Redox implementa multitarea cooperativa, donde cada proceso se ejecuta en espacio de usuario y el microkernel gestiona la planificación y sincronización básica entre tareas (?).

Sistema de archivos

`RedoxFS` es el sistema de archivos nativo, completamente escrito en Rust. Ofrece acceso concurrente, soporte para permisos y comunicación mediante el modelo de schemes.

Subsistemas de E/S y drivers

En Redox OS, los controladores de dispositivos se ejecutan como demonios en espacio de usuario. Cada driver tiene su propio namespace restringido, evitando daños al sistema principal. Esta separación aumenta la seguridad y la estabilidad del sistema (?).

Comunicación entre módulos

La comunicación se realiza mediante el sistema de schemes. Las llamadas del usuario se traducen en mensajes `SQE/CQE`, enviados entre el kernel y los servicios de usuario.

Esto asegura modularidad y aislamiento total entre componentes (?).

Planificación y temporización

El planificador emplea colas de prioridad para asignar tiempo de CPU. Busca mantener equidad, prioridad y escalabilidad con una complejidad cercana a $O(1)$ (?).

Gestión de usuarios y permisos

Los servicios `authd` y `user` manejan autenticación, control de accesos y separación de privilegios, previniendo escaladas no autorizadas (?).

Interfaz gráfica (GUI)

`Orbital` y `OrbTK` implementan el entorno gráfico de Redox. Incluyen un servidor de ventanas, gestión de eventos y un toolkit modular, todo en Rust (?).

Red y conectividad

El sistema de red, compuesto por `netstack`, `ethernetd` e `ipd`, implementa una pila TCP/IP segura en espacio de usuario, sin requerir acceso directo al kernel (?).

Gestión del sistema y servicios

Servicios como `init`, `logd` y `backgroundd` administran la inicialización, el registro de eventos y procesos en segundo plano, similares al sistema `init` de Unix (?).

Bibliotecas del sistema y compatibilidad

`relibc` y `redox_rt` ofrecen compatibilidad POSIX. Estas bibliotecas traducen funciones estándar (como `open`, `read`, `write`) a operaciones sobre schemes del sistema (?).

1.2.7 Herramientas utilizadas

El ecosistema de desarrollo de Redox OS está construido íntegramente sobre las herramientas del lenguaje Rust y un conjunto de utilidades de compilación y virtualización que garantizan portabilidad y reproducibilidad en distintos entornos.

Entre las principales herramientas empleadas se destacan:

- **Rust y Cargo:** Redox OS está escrito completamente en Rust, utilizando `cargo` como sistema de compilación y gestión de dependencias. Cada componente del sistema (kernel, drivers, librerías y servicios) se organiza como un *crate*.
- **Make y scripts de automatización:** El proceso de construcción global se gestiona mediante archivos `Makefile` y scripts en Bash, que coordinan la compilación cruzada de todos los crates y bibliotecas.
- **Podman y Docker:** Se utilizan para entornos de compilación reproducibles. Estas herramientas permiten construir el sistema dentro de contenedores sin necesidad de alterar la configuración del host.
- **QEMU:** Es el emulador principal para ejecutar y depurar Redox OS sin requerir hardware físico. Permite probar el kernel, los controladores y las aplicaciones

en un entorno controlado.

- **Git y GitLab:** Todo el código fuente está alojado en GitLab (anteriormente en GitHub). Git se usa para control de versiones, colaboración y revisión de código.
- **Herramientas de compilación cruzada (cross):** Permiten compilar Redox para distintas arquitecturas como `x86_64`, `i686` o `ARM64`, manteniendo el mismo entorno de desarrollo.

Estas herramientas no solo simplifican el desarrollo, sino que también facilitan la experimentación y el aprendizaje, ya que permiten reconstruir completamente el sistema operativo desde el código fuente, probarlo en un entorno virtual y modificar cualquier parte del kernel o de los servicios del usuario.

Corresponde a la sección 6.1–6.8 del manual oficial de Redox OS sobre el proceso de construcción y herramientas de compilación (?).

1.2.8 Nivel de complejidad y accesibilidad para estudiantes

Redox OS presenta un equilibrio notable entre complejidad técnica y accesibilidad educativa. Aunque es un sistema operativo funcional y moderno, su diseño modular en torno a un microkernel y su implementación en `Rust` facilitan el estudio y la modificación de sus componentes por parte de estudiantes o investigadores.

- **Código fuente legible y seguro:** Rust impone reglas de seguridad de memoria y control de concurrencia, reduciendo errores comunes de punteros y sincronización. Esto hace que el código de Redox sea más fácil de entender para

quienes se inician en el desarrollo de sistemas.

- **Arquitectura microkernel:** Su estructura simplificada y modular permite analizar cada subsistema de manera aislada (memoria, procesos, archivos, drivers, etc.), lo que favorece la comprensión progresiva del sistema operativo.
- **Facilidad de compilación y pruebas:** Gracias a QEMU y a los contenedores de Podman, los estudiantes pueden compilar y ejecutar Redox en sus propias máquinas sin riesgo de dañar su sistema anfitrión.
- **Documentación educativa:** El Redox OS Book ofrece guías detalladas para compilar, depurar y extender el sistema, convirtiéndolo en una herramienta pedagógica ideal para cursos de sistemas operativos o arquitectura de software.
- **Comunidad activa:** Existe una comunidad abierta de desarrolladores y estudiantes en GitLab y Matrix que brindan soporte, facilitan la colaboración y fomentan el aprendizaje conjunto.

Corresponde a las secciones 1.1–1.7 y 6.1–6.8 del manual oficial de Redox OS (??).

1.3 FlexOS

1.3.1 Nombre del proyecto o sistema operativo

El sistema operativo analizado es “FlexOS”. Según (?), su nombre proviene del término “Flexible Operating System”, reflejando su principal objetivo: ofrecer una plataforma operativa flexible y configurable que permita ajustar dinámicamente el nivel de aislamiento entre componentes. A diferencia de los sistemas tradicionales con arquitecturas rígidas (monolíticas o microkernel), FlexOS propone una arquitectura adaptable donde el desarrollador puede equilibrar seguridad, aislamiento y rendimiento según las necesidades del sistema.

1.3.2 Enlace al repositorio y/o documentación oficial

- Repositorio oficial: <https://github.com/project-flexos>
- Sitio y documentación técnica: <https://project-flexos.github.io/>
- Publicación principal: “*FlexOS: Making OS Isolation Flexible*”, presentada en EuroSys 2023 (?).

1.3.3 Objetivo del proyecto

El objetivo central de FlexOS es redefinir el equilibrio entre rendimiento y aislamiento en los sistemas operativos. Mientras los sistemas tradicionales deben optar entre un kernel monolítico rápido o un microkernel seguro pero más costoso en rendimiento, FlexOS permite ajustar el nivel de aislamiento de manera configurable. Esto se logra mediante su diseño modular, que permite que cada componente (como controladores,

memoria o IPC) se ejecute con distintos niveles de separación temporal y espacial. De esta forma, se busca proporcionar un entorno operativo capaz de adaptarse a distintos contextos: desde sistemas embebidos con recursos limitados, hasta entornos de alta seguridad o investigación académica (?).

1.3.4 Lenguaje(s) de implementación

FlexOS está implementado principalmente en C y C++, con soporte parcial en Rust para módulos experimentales que requieren mayor seguridad en memoria. Su infraestructura está construida sobre el microkernel Unikraft, al que extiende con nuevas bibliotecas y mecanismos de aislamiento. La elección de C y C++ permite mantener compatibilidad con Unikraft y con componentes de sistemas POSIX existentes, mientras que Rust se emplea en zonas críticas de seguridad o donde la verificación de memoria resulta esencial (?).

1.3.5 Arquitectura del sistema

FlexOS presenta una arquitectura modular y configurable, inspirada en los principios de los microkernels, pero con un grado de flexibilidad superior. Cada componente puede ejecutarse en un dominio de aislamiento, que define si comparte espacio de direcciones, hilos de ejecución o recursos temporales con otros módulos (?). Estos dominios pueden configurarse de tres maneras principales:

El núcleo de FlexOS gestiona la coordinación de estos dominios, la planificación del tiempo de CPU, la comunicación entre módulos (IPC) y el control de errores.

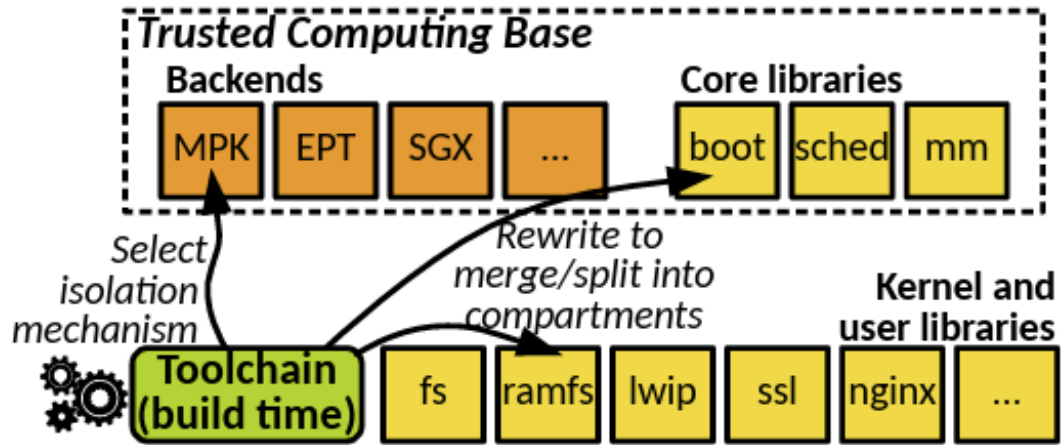


Figura 1.3: Arquitectura modular e híbrida de FlexOS.

Fuente: Obtenido de (?).

1.3.6 Componentes implementados

En FlexOS, los componentes del sistema se diseñan para maximizar el aislamiento y la modularidad, aplicando principios de sistemas operativos seguros y configurables. Su estructura permite ajustar el nivel de aislamiento (temporal y espacial) entre componentes según los requisitos de rendimiento o seguridad, convirtiéndolo en un sistema operativo híbrido altamente flexible.

Los componentes se implementan principalmente en C, C++ y Rust, utilizando microbibliotecas y servicios del kernel configurables. A continuación, se presenta una tabla con los principales subsistemas y módulos de FlexOS, junto con su función técnica principal (?).

Tabla 1.3: Funcionamiento técnico de los principales subsistemas de FlexOS

Componente / Sub-sistema	Nombre o módulo en FlexOS	Funcionamiento técnico (idea clave)
Núcleo y planificación	<code>flex_kernel</code>	Controla la ejecución y distribución del tiempo de CPU entre tareas.
Gestión de memoria	<code>mem_mgr</code> , <code>isolation_lib</code>	Maneja memoria virtual y niveles configurables de aislamiento.
Procesos y tareas	<code>task_mgr</code>	Crea y coordina tareas con comunicación segura.
Sistema de IPC	<code>ipc_core</code> , <code>shared_mem</code>	Permite intercambio de datos mediante colas y memoria compartida.
Seguridad y aislamiento	<code>sandbox</code> , <code>trusted_domain</code>	Define dominios seguros y políticas de acceso.
Gestión de E/S y drivers	<code>io_layer</code> , <code>device_mgr</code>	Ejecuta controladores aislados para proteger el kernel.
Monitoreo y fallos	<code>fault_handler</code> , <code>recovery_svc</code>	Detecta errores y reinicia módulos sin afectar el sistema.
Configuración del sistema	<code>flex_config</code>	Ajusta el nivel de aislamiento y rendimiento.
Bibliotecas de usuario	<code>libflex</code> , <code>api_runtime</code>	Facilita la comunicación segura con el kernel.

Fuente: Elaboración propia con base en (?).

Gestión de memoria

FlexOS implementa un modelo de gestión de memoria basado en *espacial isolation*, donde cada dominio o componente puede tener su propio espacio de direcciones. El sistema utiliza tanto protección por hardware (MMU) como políticas de software para garantizar independencia entre procesos y minimizar los efectos de fallos de memoria.

Procesos y multitarea

El administrador de tareas controla el ciclo de vida de los procesos mediante colas de ejecución. Los procesos pueden compartir recursos limitadamente o ejecutarse en dominios completamente aislados, dependiendo de la configuración elegida.

Comunicación entre módulos

FlexOS utiliza un sistema híbrido de comunicación: colas seguras, llamadas IPC síncronas y memoria compartida protegida. Esto permite un equilibrio entre rendimiento y seguridad, ajustable según la política de aislamiento establecida.

Gestión de E/S y drivers

Los controladores en FlexOS se ejecutan como módulos independientes o en dominios restringidos, minimizando el riesgo de fallos. Cada driver puede ser reiniciado o reemplazado sin afectar el núcleo principal.

Seguridad y aislamiento

Una de las características clave del sistema es su capacidad para reconfigurar el aislamiento entre componentes, ofreciendo niveles ajustables de separación espacial y

temporal. Esto permite ejecutar aplicaciones críticas en entornos seguros sin comprometer el rendimiento global del sistema.

1.3.7 Herramientas utilizadas (compiladores, emuladores, etc.)

FlexOS se construye sobre la infraestructura de **Unikraft**, por lo que emplea un conjunto de herramientas especializadas para la compilación, configuración y ejecución de sus módulos. El proceso de construcción del sistema operativo se basa en la cadena de herramientas de GNU Make y Kconfig, lo que permite personalizar la inclusión o exclusión de componentes del kernel según el nivel de aislamiento deseado.

Entre las herramientas más relevantes utilizadas en FlexOS se encuentran:

- **Compilador principal:** clang/LLVM, utilizado para la instrumentación del código, generación de binarios seguros y soporte para aislamiento entre dominios.
- **Infraestructura de construcción:** Unikraft Build System, que proporciona un entorno modular basado en KBuild y Makefiles para definir configuraciones del kernel y librerías.
- **Simuladores y entornos de prueba:** ejecución de instancias de FlexOS sobre QEMU y máquinas virtuales KVM, permitiendo validar configuraciones con diferentes niveles de aislamiento y medir overheads de seguridad.
- **Instrumentación y análisis:** uso de AddressSanitizer (ASan), Clang Control Flow Integrity (CFI) y trazadores de rendimiento para detectar vulnerabilidades y medir costos de comunicación entre dominios.

- **Herramientas de fuzzing y validación:** integración con **ConfFuzz**, el framework de fuzzing desarrollado por los mismos autores, usado para explorar automáticamente configuraciones de aislamiento y detectar fallos en el espacio de diseño.

1.3.8 Nivel de complejidad y accesibilidad para estudiantes

FlexOS presenta un nivel de complejidad intermedio a avanzado, dado su enfoque experimental y su dependencia de la infraestructura de Unikraft y del compilador LLVM. Aunque está diseñado como un sistema de investigación y enseñanza sobre *microkernels híbridos y aislamiento flexible*, su curva de aprendizaje es considerablemente mayor que la de sistemas educativos tradicionales como XV6 o MINIX.

Su accesibilidad depende del objetivo académico:

- Para estudiantes de pregrado, el entorno de FlexOS puede resultar complejo por su uso intensivo de compilación cruzada, configuración Kconfig y dependencias de LLVM, pero es útil para entender conceptos de modularidad y aislamiento.
- Para estudiantes de posgrado o investigadores, ofrece un entorno ideal para estudiar la relación entre *rendimiento y seguridad*, mediante experimentación con distintos niveles de compartimentalización, IPC y aislamiento de memoria.

1.4 freeRTOS

1.4.1 Nombre del proyecto o sistema operativo

FreeRTOS — Kernel de tiempo real (RTOS) de código abierto orientado a microcontroladores y microprocesadores de recursos limitados. Su historia y mantenimiento han estado ligados a Richard Barry / Real Time Engineers Ltd., y desde 2017/2018 AWS mantiene y distribuye componentes y bibliotecas asociadas (Amazon FreeRTOS/AWS FreeRTOS). (?) sistema operativo en tiempo real basado en UNIX, diseñado para ser pequeño y eficiente, ideal para sistemas embebidos y dispositivos con recursos limitados.

1.4.2 Enlace al repositorio y/o documentación oficial

- Pagina oficial: <https://www.freertos.org/>
- Enlace al repositorio oficial: <https://github.com/FreeRTOS/FreeRTOS?>
- Enlace al libro oficial: https://www.freertos.org/media/2018/161204_Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands-On_Tutorial_Guide.pdf?

1.4.3 Objetivo del proyecto

Educativo, experimental. FreeRTOS es un sistema operativo de tiempo real (RTOS) de código abierto, diseñado para ser ligero y portable en microcontroladores. Su kernel provee planificación de tareas, comunicación y temporizadores para sistemas embebidos. Además de su uso industrial, es una herramienta educativa y experimental

fundamental. Permite aprender y experimentar con conceptos de sistemas en tiempo real de manera práctica. (?, pag. 19)

1.4.4 Lenguaje(s) de implementación

El núcleo de FreeRTOS está escrito principalmente en C (para facilitar portabilidad) con pequeñas partes en ensamblador según la arquitectura (rutinas de cambio de contexto)(?),

1.4.5 Arquitectura del sistema (monolítica, microkernel, etc.)

- FreeRTOS tiene una arquitectura de tipo **microkernel**, es decir, el núcleo es muy reducido y solo gestiona lo esencial (planificación de tareas, comunicación entre procesos y gestión de memoria).
- El sistema se organiza en **capas**:
 - Una capa independiente de hardware (código común para todas las arquitecturas)
 - Una capa dependiente de hardware (abstracción para cada CPU/compilador)
 - Por encima de ellas el código de usuario (tareas e ISRs)

(?, pag. 38-40) la siguiente figura muestra las capas de software: ??.

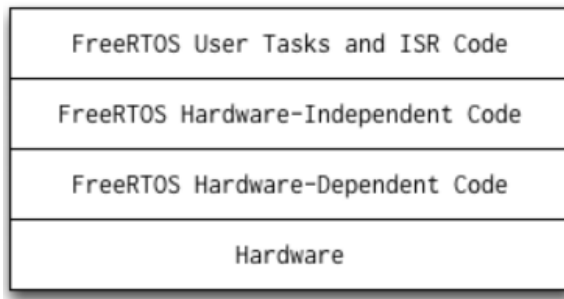


Figura 1.4: Capas de FreeRTOS (usuario/ISRs, núcleo independiente de HW, capa dependiente de HW, hardware)

Fuente: Obtenido de (?, pag. 40) *The Architecture of Open Source Applications, Volume*

II.

1.4.6 Componentes implementados (procesos, memoria, archivos, etc.)

FreeRTOS es un RTOS minimalista, por lo que implementa principalmente:

Tabla 1.4: Componentes principales de FreeRTOS

Componente / Sub-sistema	Nombre en FreeRTOS	Funcionamiento técnico (idea clave)
Gestión de tareas (threads)	tasks.c, list.c	Creación y planificación de tareas con prioridades preemptivas. Manejo de cambio de contexto en cada tick del sistema.
Comunicación / sincronización	queue.c, queue.h	Colas de mensajes FIFO, semáforos binarios y mutexes para sincronizar tareas/ISRs.
Temporización	timers.c	Temporizadores por software (delay, callbacks periódicos) con tarea de servicio propio. Tick del sistema configurable (10-1000 Hz).
Gestión de memoria	heap_1.c a heap_5.c	Múltiples esquemas de heap para asignación dinámica. Sin memoria virtual ni paginación.
Interrupciones	port*.c	Mínimo soporte en núcleo: tick del hardware y enmascaramiento de interrupciones críticas. APIs especiales para ISRs.
Eventos y temporales avanzados	event_groups.c	Grupos de flags (eventos) para sincronización múltiple entre tareas.

Gestión de tareas

Las tareas son las unidades básicas de ejecución en FreeRTOS, gestionadas por el planificador del sistema. Cada una posee su propia pila (*stack*) y estructura de control (TCB). Se crean mediante `xTaskCreate()` y se sincronizan usando los mecanismos del kernel.(?)

Comunicación / sincronización

FreeRTOS proporciona diversos mecanismos para la coordinación entre tareas e interrupciones. Entre estos se incluyen colas (`xQueueSend/xQueueReceive`), semáforos (binarios y de conteo), mutexes y grupos de eventos. Estas primitivas constituyen la base fundamental para la comunicación y sincronización dentro del sistema.(?)

Temporización

FreeRTOS incorpora temporizadores software que permiten programar la ejecución de funciones callback después de un intervalo específico o de forma periódica. Estos timers son gestionados a través de una API especializada que facilita la implementación de acciones temporizadas dentro de las aplicaciones.(?)

Gestión de memoria

FreeRTOS dispone de cinco implementaciones de memoria dinámica (`heap_1` a `heap_5`) con diferentes estrategias de asignación. Estas van desde métodos básicos hasta esquemas avanzados que gestionan la fragmentación. La selección permite adaptarse a los requerimientos específicos de cada aplicación embebida.(?)

Interrupciones

FreeRTOS gestiona las interrupciones mediante archivos `port*.c` con soporte mínimo en el núcleo. Se encarga del tick del sistema y del enmascaramiento de interrupciones críticas. Proporciona APIs específicas para una comunicación segura entre las ISRs y las tareas.(?)

Eventos y temporales avanzados

La gestión de eventos avanzados se realiza mediante `event_groups.c`, que implementa grupos de flags para sincronización múltiple entre tareas. Este mecanismo permite que varias tareas esperen o señalen múltiples condiciones de forma eficiente, facilitando la coordinación de operaciones complejas en el sistema.

1.4.7 Herramientas utilizadas (compiladores, emuladores, etc.)

Compiladores

FreeRTOS soporta prácticamente cualquier herramienta estándar del ecosistema embebido:

- GCC (`arm-none-eabi-gcc`, AVR-GCC)
- IAR Embedded Workbench
- Keil uVision (ARMCC)
- CodeWarrior
- Microchip XC

En general, se utiliza el *toolchain* propio de la plataforma objetivo.

Simuladores y Emuladores

- Puertos para Windows (Visual C++) y POSIX (Linux) que permiten ejecutar aplicaciones FreeRTOS en simulación.
- Demos en QEMU para varias arquitecturas (ARM, RISC-V, etc.).
- Demos preconfiguradas para placas reales (STM32, ESP32, etc.).

(?, pag. 40–41).

1.4.8 Nivel de complejidad y accesibilidad para estudiantes

FreeRTOS posee un kernel minimalista que facilita su aprendizaje inicial al ser más simple que sistemas como Linux. Sin embargo, su uso efectivo como sistema de tiempo real exige dominio de programación en C y arquitectura de microcontroladores para manejar concurrencia e interrupciones.

El proyecto ofrece documentación extensa y ejemplos prácticos como "blinky" que facilitan el aprendizaje. Es adecuado para estudiantes avanzados con bases en sistemas embebidos, aunque dominar sus conceptos avanzados representa un desafío formativo significativo. (?).

1.5 FreeDOS

1.5.1 Nombre del proyecto o sistema operativo

El sistema operativo FreeDOS, su nombre proviene de la idea de ofrecer una alternativa libre y abierta al sistema operativo MS-DOS, tras el anuncio de Microsoft en 1994 de interrumpir el soporte a este último (?).

1.5.2 Enlace al repositorio y/o documentación oficial

- Repositorio oficial: <https://github.com/FDOS>
- Sitio web del proyecto: <https://www.freedos.org/>

1.5.3 Objetivo del proyecto

El objetivo principal de FreeDOS es proporcionar un sistema operativo compatible con MS-DOS completamente libre, de código abierto y mantenido por la comunidad. Busca preservar la capacidad de ejecutar aplicaciones y controladores diseñados originalmente para DOS, especialmente aquellas dependientes de entornos de 16 bits (?).

1.5.4 Lenguaje(s) de implementación

FreeDOS está desarrollado principalmente en C, con secciones escritas en x86 Assembly, especialmente en el núcleo y los controladores de dispositivo. El lenguaje C se utiliza para la lógica general del sistema, mientras que el ensamblador proporciona control de bajo nivel sobre interrupciones, rutinas BIOS y gestión de hardware. Este enfoque

híbrido permite combinar eficiencia y compatibilidad con el hardware x86, tal como lo hacía MS-DOS (?).

1.5.5 Arquitectura del sistema

FreeDOS hereda la arquitectura clásica de MS-DOS, organizada en capas que separan el hardware del usuario mediante el BIOS y un núcleo DOS. El sistema se ejecuta en modo real (16 bits), operando directamente sobre la arquitectura Intel x86 sin protección de memoria ni multitarea.

Se presenta un esquema adaptado que representa la arquitectura del sistema operativo MS-DOS en que se basa FreeDOS.

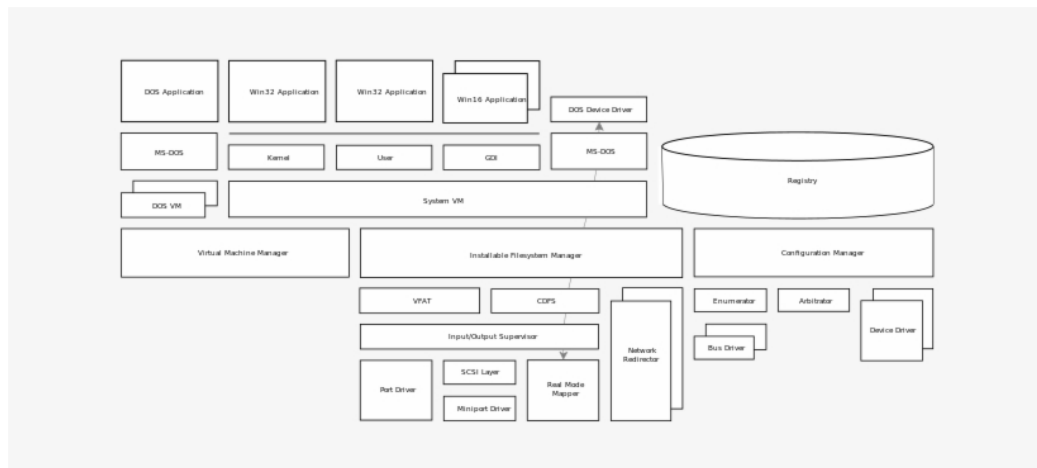


Figura 1.5: Arquitectura clásica de MS-DOS, base de FreeDOS.

Fuente: Adaptado de Microsoft Press, (?), *FreeDOS Kernel: An MS-DOS Emulator* (1996).

1.5.6 Componentes implementados (procesos, memoria, archivos, etc.)

FreeDOS es un sistema operativo de código abierto que emula el comportamiento del MS-DOS original. Está diseñado para ser compatible con aplicaciones y controladores de dispositivos escritos para MS-DOS, proporcionando una plataforma para ejecutar software antiguo en hardware moderno. A continuación, se describen los principales componentes y subsistemas implementados en FreeDOS:

Tabla 1.5: Funcionamiento técnico de los principales subsistemas de FreeDOS

Componente / Sub-sistema	Nombre en FreeDOS (archivo)	Funcionamiento técnico (idea clave)
Intérprete de comandos	COMMAND.COM / FreeCOM	Shell que carga programas, procesa scripts .BAT y maneja redirección/piping básico.
Kernel	KERNEL.SYS	Implementa servicios INT 21h (archivos, dispositivos, tiempo), gestión básica de memoria y soporte para TSRs.
Sistema de archivos	Varios drivers FAT	Soporte nativo para FAT12/FAT16/FAT32 sin VFS, con herramientas como COPY, DIR, DEL.
Gestión de memoria	HIMEM / EMM386	Modelo clásico DOS: 640KB convencional + UMB, sin protección de memoria ni paginación.
Gestión de procesos	(Parte del kernel)	Multitarea cooperativa nativa, soporte para TSRs, sin procesos aislados ni preemption.
Drivers y dispositivos	Varios .SYS / .EXE	Drivers para discos, CD-ROM, teclado/ratón; algunos actualizados para hardware moderno.

Fuente: Elaboración propia con base en la documentación técnica de FreeDOS.

1.5.6.1 Intérprete de comandos

Proporciona la interfaz de línea de comandos para interactuar con el sistema operativo. Su función principal es cargar programas ejecutables, procesar scripts por lotes (.BAT) y manejar características básicas como redirección de entrada/salida y piping entre comandos. FreeDOS incluye su propio COMMAND.COM y alternativas como FreeCOM. (? , pag. 240–242)

1.5.6.2 Kernel

Implementa las funciones centrales compatibles con MS-DOS mediante servicios de interrupción 21h para operaciones de archivos, dispositivos, tiempo y memoria. Gestiona memoria convencional y superior (UMB), soporta programas TSR (Terminate and Stay Resident) y proporciona servicios básicos de disco. El kernel de FreeDOS deriva del proyecto DOS-C y está licenciado bajo GPL v2. (?)

1.5.6.3 Sistema de archivos

Brinda soporte nativo para sistemas de archivos FAT12, FAT16 y FAT32 en modo DOS real. Permite el manejo de particiones FAT y disquetes mediante drivers, con herramientas tradicionales como COPY, DIR y DEL para operaciones básicas. A diferencia de sistemas modernos, FreeDOS no implementa un VFS ni abstracciones complejas de dispositivos. (? , pag. 30–33)

1.5.6.4 Gestión de memoria

Administra el modelo clásico de memoria DOS con espacio convencional (primeros 640 KB) y memoria superior (UMB) mediante controladores. Soporta extensiones

de memoria a través de controladores como Himem.sys y emm386.exe en entornos compatibles, pero carece de mecanismos modernos como protección de memoria o paginación presentes como en SO modernos. (? , pag. 48)

1.5.6.5 Gestión de procesos

Coordina la ejecución de programas de forma cooperativa en lugar de preemptiva. Permite el uso de programas TSR (Terminate and Stay Resident) que permanecen en memoria para proporcionar servicios, pero no implementa procesos aislados ni protección entre espacios de direcciones. Proyectos externos pueden añadir capacidades de multitarea, aunque no forman parte del kernel base. (? , pag. 50)

1.5.6.6 Drivers y dispositivos

Proporciona soporte para hardware mediante controladores para discos, CD-ROM, teclado, ratón y otro hardware clásico. algunos drivers se han actualizado para soportar hardware moderno a través de emuladores, manteniendo la compatibilidad con el ecosistema tradicional de DOS, (? , pag. 52–53)

1.5.7 Herramientas utilizadas (compiladores, emuladores, etc.)

FreeDOS puede instalarse en cualquier emulador o máquina virtual creando un disco virtual e iniciando desde el CD de instalación. Usando QEMU como ejemplo, el proceso es universal para Linux, Windows y Mac.(? , pag. 15–17)

- **Compiladores:** Utiliza principalmente GCC (GNU Compiler Collection) para la compilación del kernel y las utilidades del sistema, empleando Makefiles para

gestionar el proceso de construcción tanto nativa como cruzada.

- **Lenguajes:** Soporta principalmente lenguajes de programación C y Assembly para el desarrollo del sistema, junto con herramientas de scripting por lotes a través de archivos BAT.
- **Emuladores:** Puede ejecutarse en emuladores populares como DOSBox, QEMU y VirtualBox, lo que facilita su uso y pruebas en hardware moderno sin necesidad de equipos legacy.
- **Arquitectura compatible:** Está diseñado principalmente para arquitectura x86, con soporte para procesadores desde los 8088/8086 hasta sistemas modernos a través de emulación.

1.5.8 Nivel de complejidad y accesibilidad para estudiantes

FreeDOS enseña conceptos históricos y de bajo nivel de PC (BIOS, boot, interrupciones, FAT); es complementario: excelente para aprendizaje de hardware/arranque y compatibilidad, menos apropiado para enseñar técnicas modernas de diseño de SO (ej. virtual memory, protección por procesos).(?)

- **Complejidad técnica:** Abarca desde nivel básico (comandos DOS, scripting BAT, programación con interrupciones) hasta avanzado (desarrollo de kernel, drivers, porting de hardware).
- **Accesibilidad educativa:** Muy alta para laboratorios de arquitectura de computadores, permitiendo estudiar el proceso de arranque (boot sector), interrupciones BIOS/DOS (int 10h, int 21h) y programación en entornos limitados.

Capítulo 2

Comparación técnica

La tabla siguiente presenta una comparación técnica de varios sistemas operativos destacados, considerando aspectos clave como arquitectura, lenguaje de programación, tamaño del kernel, soporte de hardware, documentación y comunidad de usuarios y desarrolladores.

Capítulo 3

Análisis crítico

Referencias